

Start coding or [generate](#) with AI.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
df = pd.read_csv("FinalDataset.csv")
```

```
df.head()
```

	Store	Department	Date	Weekly_Sales	Is_Holiday	Temperature	Fuel_Price
0	1	1	05-02-2010	24924.50	False	42.31	2.572
1	1	2	05-02-2010	50605.27	False	42.31	2.572
2	1	3	05-02-2010	13740.12	False	42.31	2.572
3	1	4	05-02-2010	39954.04	False	42.31	2.572
4	1	5	05-02-2010	32229.38	False	42.31	2.572

```
pd
```

```
<module 'pandas' from '/usr/local/lib/python3.12/dist-packages/pandas/__init__.py'>
```

```
df.isnull().sum()
```

	0
Store	0
Department	0
Date	0
Weekly_Sales	0
Is_Holiday	0
Temperature	0
Fuel_Price	0
CPI	0
Unemployment	0
Type	0
Size	0
Year	0
Month	0
Quarter	0
Week_of_Year	0

dtype: int64

Start coding or [generate](#) with AI.

```
df.fillna(0, inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 421570 entries, 0 to 421569
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Store            421570 non-null  int64
1   Department       421570 non-null  int64
2   Date             421570 non-null  object
3   Weekly_Sales     421570 non-null  float64
4   Is_Holiday       421570 non-null  bool
5   Temperature      421570 non-null  float64
6   Fuel_Price       421570 non-null  float64
7   CPI              421570 non-null  float64
8   Unemployment     421570 non-null  float64
9   Type             421570 non-null  object
10  Size             421570 non-null  int64
11  Year             421570 non-null  int64
```

```
12 Month          421570 non-null  int64
13 Quarter        421570 non-null  int64
14 Week_of_Year   421570 non-null  int64
dtypes: bool(1), float64(5), int64(7), object(2)
memory usage: 45.4+ MB
```

```
X = df[
    [
        "Temperature",
        "Fuel_Price",
        "CPI",
        "Unemployment",
        "Size"
    ]
]
```

```
y = df["Weekly_Sales"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
▼ LinearRegression ⓘ ?
LinearRegression()
```

```
predictions = model.predict(X_test)
```

```
comparison = pd.DataFrame({
    "Actual": y_test,
    "Predicted": predictions
})

comparison.head(10)
```

	Actual	Predicted	
272342	50932.42	21530.067385	
176581	3196.12	14257.839840	
354212	10125.03	6615.184560	
281444	3311.26	12095.050667	
124208	6335.65	24174.208180	
123901	8971.23	24902.322139	
111563	1575.35	13650.695612	
93836	17308.45	16120.994924	
102266	5763.39	22186.010076	
254986	17034.57	18210.369792	

Next steps:

[Generate code with comparison](#)[New interactive sheet](#)

```

mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, predictions)

```

```

print("Mean Absolute Error :", mae)
print("Mean Squared Error :", mse)
print("Root Mean Squared Error :", rmse)
print("R2 Score :", r2)

```

```

Mean Absolute Error : 14517.243870422344
Mean Squared Error : 488937653.9443294
Root Mean Squared Error : 22111.934649512903
R2 Score : 0.06245747548727698

```

```

plt.figure(figsize=(8,6))

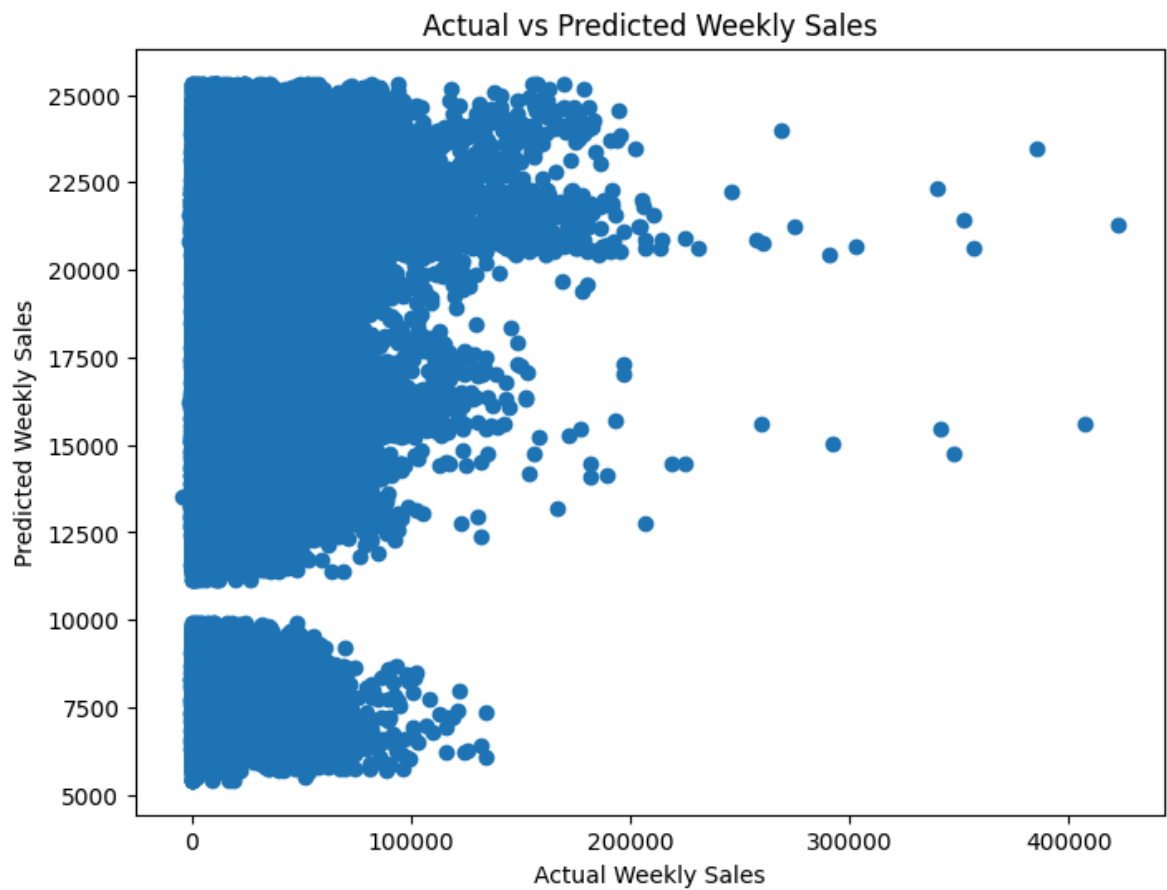
plt.scatter(y_test, predictions)

plt.xlabel("Actual Weekly Sales")
plt.ylabel("Predicted Weekly Sales")

plt.title("Actual vs Predicted Weekly Sales")

plt.show()

```



```
residuals = y_test - predictions

plt.figure(figsize=(8,6))

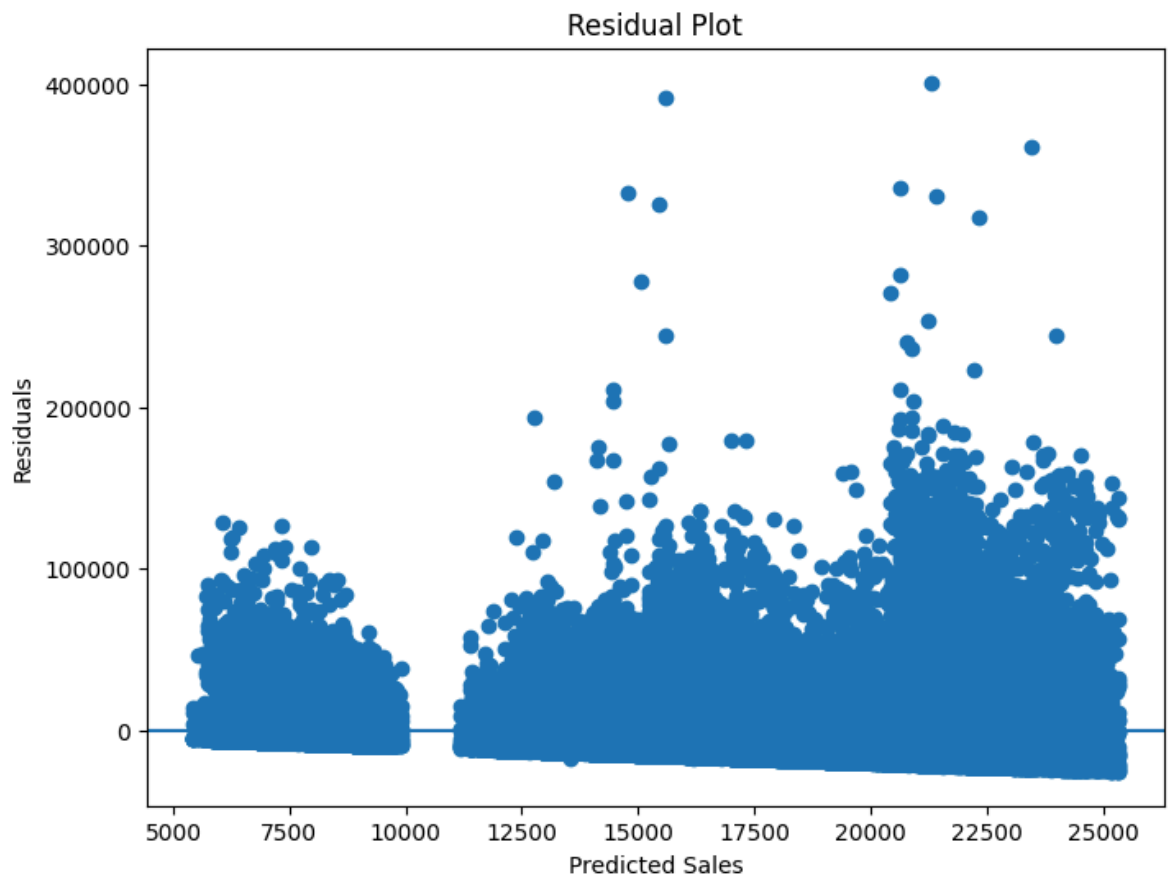
plt.scatter(predictions, residuals)

plt.axhline(y=0)

plt.xlabel("Predicted Sales")
plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.show()
```



```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})
```

coefficients

	Feature	Coefficient	
0	Temperature	27.956012	
1	Fuel_Price	-499.187831	
2	CPI	-18.728528	
3	Unemployment	-260.372714	
4	Size	0.090162	

Next steps:

[Generate code with coefficients](#)

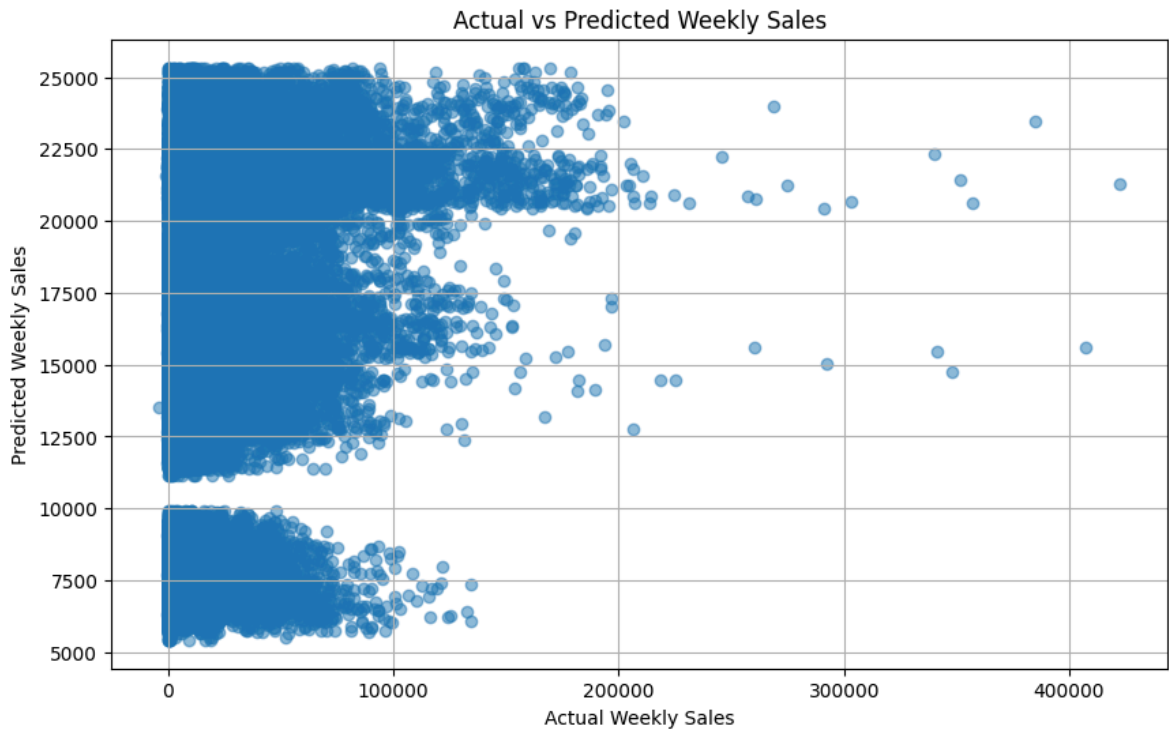
[New interactive sheet](#)

```
comparison.to_csv("Predicted_Sales.csv", index=False)
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.scatter(y_test, predictions, alpha=0.5)
plt.xlabel("Actual Weekly Sales")
plt.ylabel("Predicted Weekly Sales")
plt.title("Actual vs Predicted Weekly Sales")
plt.grid(True)

plt.show()
```



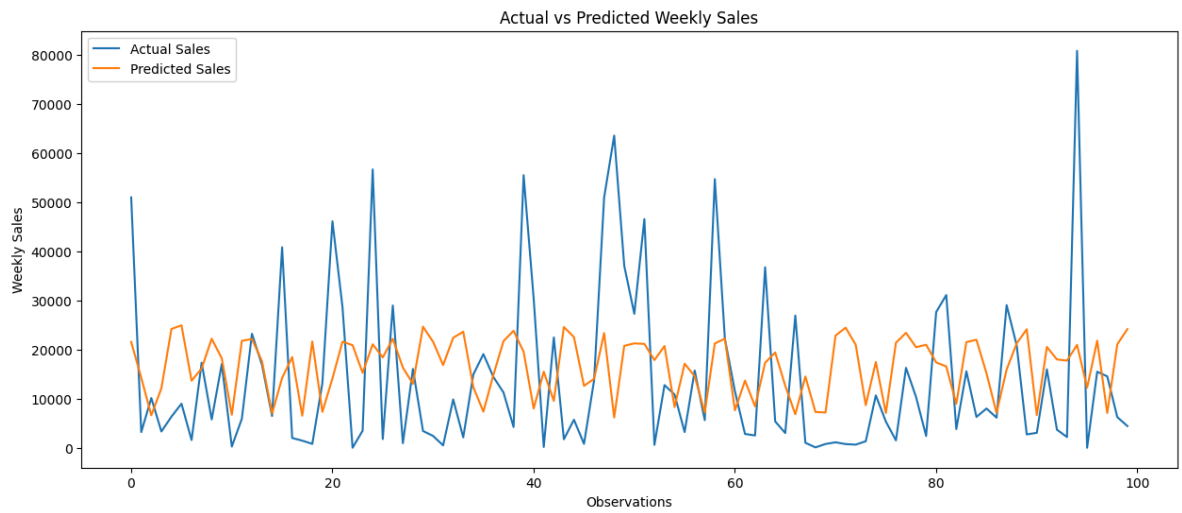
```
plt.figure(figsize=(15,6))

plt.plot(y_test.values[:100], label="Actual Sales")
plt.plot(predictions[:100], label="Predicted Sales")

plt.title("Actual vs Predicted Weekly Sales")
plt.xlabel("Observations")
plt.ylabel("Weekly Sales")

plt.legend()

plt.show()
```



```
residuals = y_test - predictions

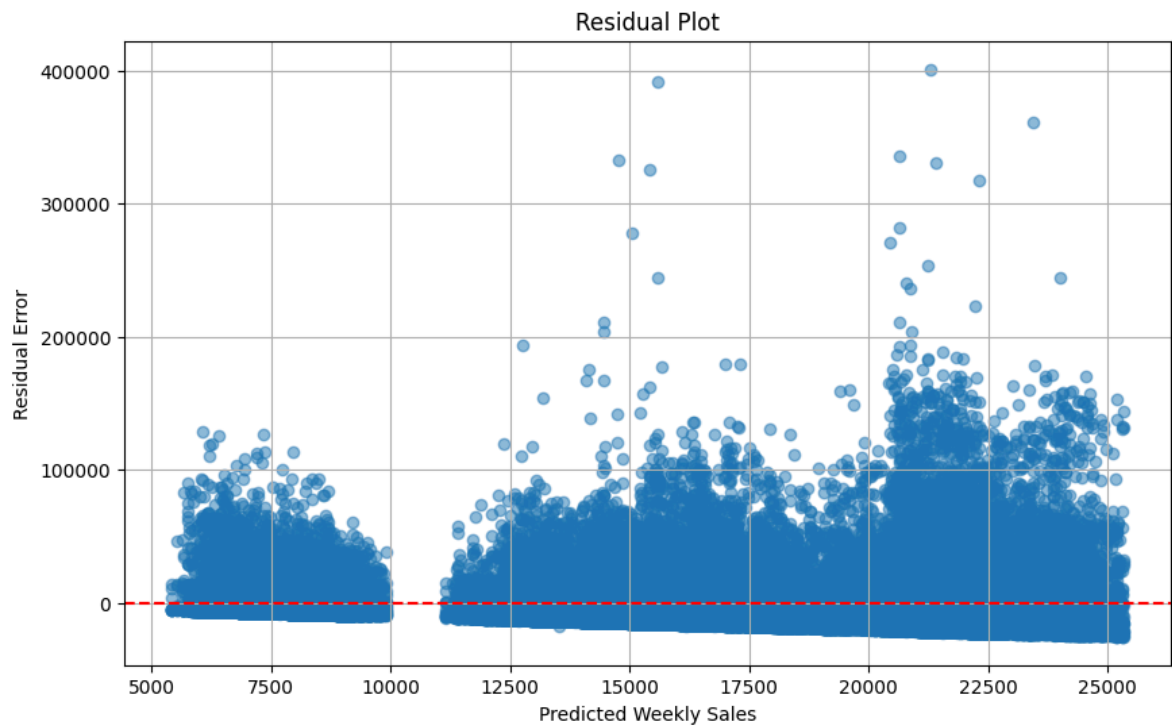
plt.figure(figsize=(10,6))
plt.scatter(predictions, residuals, alpha=0.5)

plt.axhline(y=0, color='red', linestyle='--')

plt.xlabel("Predicted Weekly Sales")
plt.ylabel("Residual Error")
plt.title("Residual Plot")

plt.grid(True)

plt.show()
```

```
import pandas as pd

coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

coefficients = coefficients.sort_values(by='Coefficient', ascending=False)

print(coefficients)
```

	Feature	Coefficient
0	Temperature	27.956012
4	Size	0.090162
2	CPI	-18.728528
3	Unemployment	-260.372714
1	Fuel_Price	-499.187831

```
plt.figure(figsize=(8,5))

plt.bar(coefficients['Feature'], coefficients['Coefficient'])

plt.title("Feature Importance")
plt.xlabel("Features")
plt.ylabel("Coefficient")
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```

